

Appunti del corso di Basi di dati

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis Matteo Drago

Indice

1	Tecnologie per le basi di dati	1
1.1	Architettura transazionale di un DBMS	1
1.2	Transazione	2
1.2.1	Sintassi	2
1.2.2	Proprietà delle transazioni	3
1.3	Architettura di un DBMS	3
2	Linguaggio SQL	6
2.1	Clausola SELECT	7
2.2	Clausola FROM	9
2.3	Clausola WHERE	10
2.3.1	Operatore LIKE	11
2.3.2	Operatore SIMILAR TO	12
2.3.3	Operatore BETWEEN	13
2.3.4	Operatore IN	14
2.3.5	Operatore IS NULL	14
2.4	Clausola DISTINCT	15
2.5	Clausola JOIN	16
2.5.1	Clausola INNER JOIN	17
2.5.2	Clausola LEFT OUTER JOIN	18
2.5.3	Clausola RIGHT OUTER JOIN	19
2.5.4	Clausola FULL OUTER JOIN	19
2.6	Clausola AS	19
2.7	Clausola ORDER BY	20
2.8	Operazioni insiemistiche	22
2.8.1	Operatore UNION	22
2.8.2	Operatore INTERSECT	24
2.8.3	Operatore EXCEPT	25
2.9	Operatori aggregati	25
2.9.1	Operatore COUNT	25
2.9.2	Operatore SUM	26
2.9.3	Operatore MAX/MIN	27
2.9.4	Operatore AVG	27
2.10	Clausola GROUP BY	28
2.10.1	Clausola HAVING	29
2.11	Interrogazioni nidificate	30

2.11.1	Clausola WHERE	30
2.11.2	Clausola EXISTS	31
2.11.3	Clausola SELECT	32
2.11.4	Clausola FROM	32
3	Strutture fisiche e di accesso ai dati	34
3.1	Gestore del buffer	34
3.1.1	Buffer	35
3.1.2	Politica di gestione della memoria	35
3.1.3	Gestione delle pagine	35
3.1.4	Primitive per la gestione del buffer	36
3.1.4.1	Primitiva fix	36
3.1.4.2	Primitiva unfix	36
3.1.4.3	Primitiva setDirty	36
3.1.4.4	Primitiva force	36
3.1.4.5	Primitiva flush	37
3.2	Gestore dell'affidabilità	37
3.2.1	File di LOG	37
3.2.2	Regole di scrittura sul file di LOG	38
3.2.3	Guasti	39
3.3	Gestore dei metodi di accesso	41
3.3.1	Metodi di accesso	41
3.3.2	Organizzazione di una pagina	42
3.3.3	Operazioni sulle pagine	43
3.3.4	Strutture primarie per l'organizzazione dei file	43
3.3.4.1	Struttura sequenziale	43
3.3.4.1.1	Struttura sequenziale disordinata	43
3.3.4.1.2	Struttura sequenziale ad array	44
3.3.4.1.3	Struttura sequenziale ordinata	44
3.3.4.1.4	Indice	45
3.3.4.2	Struttura ad albero	48
3.3.4.2.1	Struttura	48
3.3.4.2.2	Vincoli di riempimento	49
3.3.4.2.3	Operazione: ricerca con chiave K	50
3.3.4.2.4	Operazione: inserimento	51
3.3.4.2.5	Operazione: cancellazione	52
3.3.4.3	Struttura ad accesso calcolato (hash)	53
3.3.4.3.1	Operazione: ricerca con chiave K	54
3.3.4.3.2	Collisioni	54
4	Esecuzione concorrente di transazioni	56
4.1	Tipologie di anomalie	56
4.1.1	Perdita di aggiornamento	57
4.1.2	Lettura inconsistente	57
4.1.3	Lettura sporca	58
4.1.4	Aggiornamento fantasma	58
4.1.5	Inserimento fantasma	59
4.2	Schedule	59
4.2.1	Schedule seriale	60

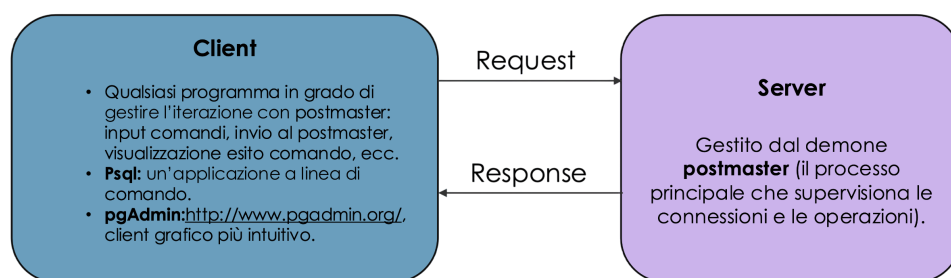
4.3	Tecniche di gestione della concorrenza	60
4.3.1	View-equivalenza	60
4.3.2	Conflict-equivalenza	65
4.3.3	Locking a due fasi stretto	70
4.3.4	Blocco critico (deadlock)	72
4.3.5	Gestione della concorrenza in SQL	73
5	Ottimizzazione di interrogazioni	74
5.1	Ottimizzazione dipendente dai metodi di accesso	74
5.1.1	Scansione sequenziale	75
5.1.2	Ordinamento	75
5.1.3	Accesso diretto tramite indice	76
5.1.4	Join	77
5.1.4.1	Nested loop join	77
5.1.4.2	Nested loop join con indice	78
5.1.4.3	Block nested loop join	79
5.1.4.4	Merge scan join	80
5.1.4.5	Hash-based join	80
5.2	Scelta del piano esecuzione	81
6	Laboratorio	82
6.1	Comando CREATE TABLE	82
6.1.1	Identificatori	83
6.1.2	Domini elementari	83
6.1.2.1	Tipo carattere	83
6.1.2.2	Tipo boolean	84
6.1.2.3	Tipo numerico esatto	84
6.1.2.4	Tipo numerico approssimato	85
6.1.2.5	Istanti temporali	85
6.1.2.6	Intervalli temporali	85
6.1.3	Domini definiti dall'utente	86
6.1.4	Vincoli intrarelazionali	86
6.1.5	Vincoli interrelazionali	87
6.2	Modifica degli schemi	88
6.2.1	Comando ALTER TABLE	88
6.3	Comando DELETE e DROP TABLE	89
6.4	Comando INSERT INTO e UPDATE	89
6.5	Selezione	90
6.5.1	Clausola SELECT	91
6.5.1.1	Target List	91
6.5.1.2	Concatenazione stringhe	92
6.5.2	Clausola FROM	92
6.5.2.1	Uso di alias	93
6.5.2.2	Target List	93
6.5.3	Clausola WHERE	93
6.5.3.1	Operatore LIKE	93
6.5.3.2	Operatore SIMILAR TO	94
6.5.3.3	Operatore BETWEEN	94
6.5.3.4	Operatore IN	94

	6.5.3.5 Operatore IS NULL	95
	6.5.3.6 Operatore ORDER BY	95
	6.5.3.7 Operatori di aggregazione	95
6.6	Esempi di interrogazioni	96
6.6.1	Interrogazione con raggruppamento: Group by	96
6.6.2	Interrogazione con JOIN	98
6.6.3	Interrogazioni con LEFT OUTER JOIN	98
6.6.4	Interrogazioni con RIGHT OUTER JOIN	98
6.6.5	Interrogazioni con FULL OUTER JOIN	99
6.7	Interrogazioni nidificate e insiemistiche	99
6.7.1	Clausola FROM	99
6.7.2	Clausola WHERE	99
	6.7.2.1 Operatore ANY/SOME	100
	6.7.2.2 Operatore ALL	100
	6.7.2.3 Operatore IN	101
	6.7.2.4 Operatore EXIST	101
6.7.3	Le viste	104
6.8	Indici	105
6.8.1	Creazione indice	106
6.8.2	Caso pratico	107
6.8.3	Tipi di indice	108
	6.8.3.1 B-tree	108
	6.8.3.2 Hash	108
	6.8.3.3 Multi-attributo	108
	6.8.3.4 Indici di espressioni	109
6.8.4	Costo e Pratiche d'uso	109
	6.8.4.1 Explain con due nodi	110
	6.8.4.2 Explain con due nodi – AND con un solo indice	111
	6.8.4.3 Esempi d'uso	111
6.8.5	Explain versione avanzata	113
6.9	Introduzione al controllo di concorrenza in SQL	114
6.9.1	Transazioni	114
	6.9.1.1 Controllo concorrenza funzionamento	115
6.9.2	Livelli di isolamento	116
	6.9.2.1 Sintassi	117
	6.9.2.2 Livello di isolamento: Read Committed	117
	6.9.2.3 Livello di isolamento Repeatable Read	119
	6.9.2.4 Livello di isolamento: Repeatable Read	120
	6.9.2.5 Serializable: avvertenze	121
	6.9.2.6 Cosa fare in caso di failure:	121
6.9.3	Lock Espliciti	122

Laboratorio

PostgreSQL è un **R**elational **D**ata **B**ase **M**anagement **S**ystem con alcune funzionalità orientate agli oggetti.

L'interazione tra un utente (programmatore/utente finale) e le basi di dati gestite da PostgreSQL avviene secondo il modello client-server.



6.1 Comando CREATE TABLE

Il comando `CREATE TABLE` definisce uno schema di relazione e ne crea un'istanza vuota, specificando attributi, domini e vincoli.

La sintassi è:

```
1 CREATE TABLE tabella(  
2     attributo dominio [valoreDefault] {vincolo},  
3     attributo dominio [valoreDefault] {vincolo},  
4     {vincoloTabella}  
5 );
```

dove:

- ▷ `[]`: indicano parti opzionali
- ▷ `[valoreDefault]`: indica un eventuale valore di default
- ▷ `{}`: indica un elemento che può essere ripetuto 0 o più volte
- ▷ `{vincolo}`: indica eventuali (0 o più) vincoli per ogni attributo

Utilizzo di CREATE TABLE

```
1 CREATE TABLE Impiegato (  
2   Matricola CHARACTER(6) PRIMARY KEY,  
3   Nome VARCHAR(20) NOT NULL,  
4   Cognome VARCHAR(20) NOT NULL,  
5   Dipart VARCHAR(15),  
6   Stipendio NUMERIC(9) DEFAULT 0,  
7   FOREIGN KEY(Dipart)  
8     REFERENCES Dipartimento(NomeDip),      -- vincolo di tabella  
9   UNIQUE (Cognome, Nome)                    -- vincolo di tabella  
10 );
```

6.1.1 Identificatori

Definizione: identificatore

Un **identificatore** è una sequenza di caratteri che identifica un elemento del database (tabella, attributo, vincolo, ecc.).

Gli identificatori, in SQL, devono rispettare alcune regole:

- ◇ iniziare con una lettera (UTF-8) o un underscore (_)
- ◇ possono contenere lettere, numeri (0-9) o underscore (_)
- ◇ non possono essere parole chiave riservate a SQL

Nota

Gli identificatori **non sono case sensitive!** (idPersona = idpersona)

6.1.2 Domini elementari

Definizione: domini elementari

I domini elementari (predefiniti) sono:

- ▷ **caratteri**: singoli caratteri o stringhe anche di lunghezza variabile
- ▷ **bit**: bit singoli (booleani) o stringhe di bit
- ▷ **numeri esatti e approssimati**
- ▷ **istanti temporali**
- ▷ **intervalli temporali**

6.1.2.1 Tipo carattere

Permette di rappresentare singoli caratteri oppure stringhe.

Un carattere o stringa di caratteri sono rappresentati tra apici ('): 'a', 'Questa è una stringa'.

La sintassi è:

```
1 CHARACTER [VARYFING] [(lunghezza)]
```

dove:

- ▷ lunghezza: può essere fissa o variabile
 - ◇ CHARACTER: carattere singolo
 - ◇ CHARACTER(20): stringa di lunghezza fissa (20 caratteri).
Se la stringa è più corta di 20 caratteri, viene riempita con spazi a destra
 - ◇ CHARACTER VARYFING(20) o VARCHAR(20): stringa di lunghezza variabile (max. 20 caratteri)
 - ◇ TEXT: stringa di lunghezza variabile senza limite fissato

6.1.2.2 Tipo boolean

Permette di rappresentare valori booleani (true o false), più lo stato nullo.

La sintassi è:

```
1 BOOLEAN
```

La rappresentazione dei valori booleani può essere:

- TRUE o FALSE
- t o f
- 1 o 0
- yes o no
- NULL

6.1.2.3 Tipo numerico esatto

Permettono di rappresentare valori esatti (interi o con una parte decimale di lunghezza prefissata):

- ▷ **valori interi:**
 - ◇ SMALLINT: valori interi (2 bytes: da -32.768 (215) a +32.767 (215-1))
 - ◇ INTEGER: valori interi (4 bytes: da -2147483648 (231) to +2147483647 (231-1))
- ▷ **valori decimali:**
 - ◇ NUMERIC [(precisione [, scala])]:
 - precisione: numero totale di cifre significative (cifre a sinistra e a destra della virgola)
 - scala: numero di cifre dopo la virgola (vuoto = 0)
 - ◇ DECIMAL [(precisione [, scala])]: equivalente al precedente

Esempio: tipo numerico esatto

NUMERIC(5,2) permette di rappresentare valori come 100.01, 100.999 (arrotondato a 101.00), ma non valori come 1000.01

6.1.2.4 Tipo numerico approssimato

Permettono di rappresentare valori in virgola mobile:

- ◇ REAL: precisione di 6 cifre decimali
- ◇ DOUBLE PRECISION: precisione di 15 cifre decimali

Ciascun numero è rappresentato da una coppia di valori: **mantissa** (numero frazionario) + **esponente** (numero intero).

Il valore si ottiene moltiplicando la mantissa per la potenza di 10 con grado pari all'esponente:

- numero 0.17E16 corrisponde a $1,7 \times 10^{15}$
- numero -0.4E-6 corrisponde a -4×10^{-7}

6.1.2.5 Istanti temporali

Permettono di rappresentare istanti di tempo.

- ◇ DATE: istanti del tipo (anno, mese, giorno)
Si rappresenta tra apici (es. '2025-12-01') e lo standard ISO prescrive il formato YYYY-MM-DD
- ◇ TIME [(precisione)] [WITH TIME ZONE]: istanti del tipo (hour, minute, second)
 - precisione: numero cifre decimali per rappresentare frazioni del secondo
 - WITH TIME ZONE: permette di specificare anche [+/-] hour: minute, che rappresentano la differenza con l'ora di Greenwich, o nomeFusoOrario
- ◇ TIMESTAMP [(precisione)] [WITH TIME ZONE]: date + time

6.1.2.6 Intervalli temporali

Rappresenta un intervallo di tempo.

La sintassi è:

```
1 INTERVAL PrimaUnitaTempo [TO UltimaUnitàTempo]
```

dove:

- ▷ PrimaUnitàDiTempo e UltimaUnitàDiTempo: definiscono le unità di misura che devono essere utilizzate, dalla più precisa alla meno precisa

L'insieme delle unità di misura è diviso in due insiemi distinti:

- Year-Month
- Day-Hour-Minute-Second

6.1.3 Domini definiti dall'utente

Per creare un dominio si esegue il comando:

```
1 CREATE DOMAIN nome AS tipoBase [valoreDefault][vincolo]
```

dove:

- ▷ CREATE DOMAIN: definisce un dominio, utilizzabile in definizioni di relazioni, anche con vincoli e valori di default

Il dominio può essere definito a partire da un dominio elementare oppure da un altro dominio definito dall'utente.

Comando CREATE DOMAIN

```
1 CREATE DOMAIN giorniSettimana AS CHARACTER(3)
2 CHECK(VALUE IN('LUN', 'MAR', 'MER', 'GIO', 'VEN', 'SAB', 'DOM'))
3 ;
4 CREATE DOMAIN Voto AS SMALLINT DEFAULT NULL
5 CHECK (value >= 18 AND value <= 30);
```

6.1.4 Vincoli intrarelazionali

I vincoli intrarelazionali sono vincoli che si applicano a livello di relazione, cioè a una singola tabella.

I principali vincoli intrarelazionali sono:

- ◇ NOT NULL: valore nullo non è ammesso per l'attributo, quindi il valore deve sempre essere specificato, o si assegna un valore di default
- ◇ UNIQUE: definisce (super)chiavi

```
1 Nome VARCHAR(20),
2 Cognome VARCHAR(20),
3 UNIQUE(Nome, Cognome)
4 --impone che non ci siano due righe che abbiano uguale
5 --sia il nome che il cognome
```

```
1 Nome VARCHAR(20) UNIQUE,
2 Cognome VARCHAR(20) UNIQUE
3 --impone che non ci siano due righe che abbiano lo stesso nome
4 --o lo stesso cognome
```

- ◇ PRIMARY KEY: chiave primaria (una sola, implica NOT NULL)

```
1 matricola CHAR(6) PRIMARY KEY;
2 --se unico componente della chiave primaria
```

```

1 nome VARCHAR(20),
2 cognome VARCHAR(20),
3 PRIMARY KEY(nome, cognome)
4 --se la chiave primaria ha piu' attributi

```

- ◇ CHECK: specifica un vincolo generico che deve soddisfare le tuple della tabella. Viene soddisfatto se la sua espressione è vera o nulla

```

1 CREATE TABLE Impiegato(
2 matricola CHAR(6) PRIMARY KEY,
3 nome VARCHAR(20) NOT NULL,
4 cognome VARCHAR(20) NOT NULL,
5 qualifica VARCHAR(20),
6 stipendio NUMERIC(8,2) DEFAULT 500.00 NOT NULL
7 CHECK(stipendio >= 0.0),      --check di attributo
8 UNIQUE(cognome, nome),
9 CHECK(nome <> cognome)        --check di tabella
10 )

```

6.1.5 Vincoli interrelazionali

Un vincolo di integrità referenziale (interrelazionale) crea un legame tra i valori di un attributo (o di un insieme di attributi) A della tabella corrente (interna/slave) e i valori di un attributo (o di un insieme di attributi) B di un'altra tabella (esterna/master).

Il vincolo impone che:

- ◇ in ogni tupla della tabella interna, il valore di A, se diverso dal valore nullo, sia presente tra i valori di B nella tabella esterna
- ◇ l'attributo B della tabella esterna deve essere soggetto a un vincolo UNIQUE (o PRIMARY KEY)

Nota

L'attributo (o insieme di attributi) B può anche non essere la chiave primaria, ma deve essere identificante per le tuple della tabella esterna.

REFERENCES e FOREIGN KEY permettono di definire vincoli di integrità referenziale.

Per singoli attributi basta utilizzare il costrutto REFERENCES.

```

1 CREATE TABLE Impiegato (
2   Matricola CHARACTER(6) PRIMARY KEY,
3   Nome CHARACTER VARYING(20) NOT NULL,
4   Cognome CHARACTER VARYING(20) NOT NULL,
5   Dipart CHARACTER VARYING(15),
6   REFERENCES Dipartimento(NomeDip),  --attributo chiave (singolo) di
   tabella esterna
7   Ufficio NUMERIC(3),
8   Stipendio NUMERIC(9) DEFAULT 0,
9   UNIQUE(Cognome, Nome)
10 )

```

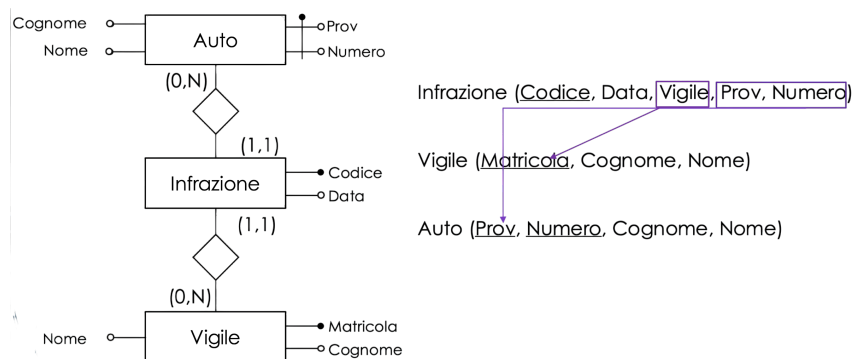
Per attributi multipli, si usa il costrutto FOREIGN KEY alla fine della definizione degli attributi, seguito da REFERENCES.

```

1 CREATE TABLE Impiegato (
2   Matricola CHARACTER(6) PRIMARY KEY,
3   Nome CHARACTER VARYING(20) NOT NULL,
4   Cognome CHARACTER VARYING(20) NOT NULL,
5   Dipart CHARACTER VARYING(15),
6   REFERENCES Dipartimento(NomeDip),
7   Ufficio NUMERIC(3),
8   Stipendio NUMERIC(9) default 0,
9   FOREIGN KEY(Nome, Cognome)
10    REFERENCES Anagrafica (Nome, Cognome) --tabella esterna con
    attributi chiave
11 )

```

Vediamo un esempio più concreto:



```

1 CREATE TABLE Infrazione(
2   Codice CHAR(6) PRIMARY KEY,
3   Data DATE NOT NULL,
4   Vigile INTEGER NOT NULL
5   REFERENCES Vigile(Matricola),
6   Provincia CHAR(2),
7   Numero CHAR(6) ,
8   FOREIGN KEY(Provincia, Numero)
9    REFERENCES Auto(Provincia, Numero)
10 )

```

6.2 Modifica degli schemi

- ▷ ALTER: effettua modifiche (a domini, tabelle...), ad esempio aggiungere una colonna a una relazione
- ▷ DROP DOMAIN: permette di rimuovere i domini
- ▷ DROP TABLE: cancella una tabella

6.2.1 Comando ALTER TABLE

Il costrutto ALTER permette di:

- aggiungere un nuovo attributo (ADD COLUMN):

```
1 ALTER TABLE tabella
2 ADD COLUMN attributo tipo;
3
4 ALTER TABLE impiegato
5 ADD COLUMN stipendio NUMERIC(8,2);
```

- rimuovere un attributo (DROP COLUMN):

```
1 ALTER TABLE tabella
2 DROP COLUMN attributo;
3
4 ALTER TABLE impiegato
5 DROP COLUMN stipendio;
```

- modificare valore di default di un attributo (ALTER COLUMN):

```
1 ALTER TABLE tabella
2 ALTER COLUMN attributo { SET DEFAULT valore | DROP DEFAULT };
3
4 ALTER TABLE impiegato
5 ALTER COLUMN stipendio SET DEFAULT 1000.00;
```

6.3 Comando DELETE e DROP TABLE

Le tuple di una tabella vengono cancellate con il comando DELETE:

```
1 DELETE FROM tabella [ WHERE condizione ];
```

dove:

- ▷ condizione: espressione booleana che seleziona quali tuple cancellare

Nota

Se non viene specificata la condizione WHERE, verranno cancellate tutte le tuple.

```
1 DELETE FROM impiegato WHERE matricola = 'A001';
```

Una tabella viene cancellata con il comando DROP TABLE:

```
1 DROP TABLE tabella;
```

6.4 Comando INSERT INTO e UPDATE

Il comando INSERT INTO permette di inserire valori all'interno di una tupla:

```

1 INSERT INTO Tabella [ ( Attributi ) ]
2 VALUES( Valori )
3
4 --oppure
5
6 INSERT INTO Tabella [ ( Attributi )]
7 (SELECT ...)

```

Utilizzo di INSERT INTO

```

1 INSERT INTO Museo(nome, citta, indirizzo, numeroTelefono,
   giornoChiusura, prezzo)
2 VALUES('Arena', 'Verona', 'piazza Bra', '045 8003204', 'MARTEDI',
   20);

```

Il comando UPDATE permette di aggiornare i valori all'interno di una tupla:

```

1 UPDATE NomeTabella
2 SET Attributo = < Espressione | SELECT ... | NULL | DEFAULT >
3 [ WHERE Condizione ]

```

Utilizzo di INSERT INTO

```

1 UPDATE Museo SET sitoInternet = 'www.ciao.it'
2 WHERE citta = 'Verona';

```

6.5 Selezione

La struttura base del comando di selezione è:

```

1 SELECT [ DISTINCT ]
2 [ * | expression [[ AS] output_name ] [,
3 ...] ]
4 FROM from_item [, ...]
5   [ WHERE condition ]
6   [ GROUP BY grouping_element [, ...] ]
7   [ HAVING condition [, ...] ]
8   [ { UNION | INTERSECT | EXCEPT } [ DISTINCT
9   ] other_select ]
10  [ ORDER BY expression [ ASC | DESC | USING
11  operator ]]

```

6.5.1 Clausola SELECT

```
1 SELECT [ DISTINCT ]
2 [{* | expression [[ AS ] output_name ]] [, ...]]
```

dove:

- ▷ *: abbreviazione per indicare tutti gli attributi delle tabelle
- ▷ expression: espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo)
- ▷ output_name: nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione expression nella relazione risultato.
- ▷ DISTINCT: se presente richiede l'eliminazione delle tuple duplicate

L'esecuzione di una SELECT produce una relazione che:

- ha come schema tutti gli attributi elencati nella clausola SELECT
- ha come contenuto tutte le tuple ottenute proiettando sugli attributi indicati nella SELECT provenienti dalle tabelle del FROM (prodotto cartesiano), limitate a quelle che soddisfano le eventuali condizioni specificate nelle clausole WHERE, GROUP BY e HAVING

6.5.1.1 Target List

```
1 SELECT *
2 FROM Tabella
```

Selezione di tutti gli attributi delle tabelle indicate nella clausola FROM

```
1 SELECT Attributo1,
2        Attributo2
3 FROM Tabella
```

Selezione di tutti gli attributi delle tabelle indicate nella clausola FROM

Esempio

```
1 SELECT Nome, Reddito
2 FROM Persone
3 WHERE Eta < 30;
```

Nome	Reddito
Andrea	21
Aldo	15
Filipo	30

Figura 6.1: selezione solo delle tuple dove è rispettata la condizione

6.5.1.2 Concatenazione stringhe

L'operatore di concatenazione `||` è un operatore scalare (o operatore di riga).

Lavora su una singola riga alla volta.

Se la tua tabella ha 100 righe, il costrutto `SELECT titolo || ' - ' || città` calcolerà e restituirà 100 risultati testuali separati, uno per ogni riga.

Nota

Non è un'esclusiva di `SELECT`, può infatti essere utilizzato anche nella clausola `WHERE` e nella clausola `ORDER BY`.

La concatenazione all'interno della clausola `WHERE` serve per filtrare i dati:

```
1 WHERE nome || ' ' || cognome = 'Mario Rossi'
2 --Trova la riga in cui l'unione di nome e cognome da esattamente
3 --'Mario Rossi'
```

La concatenazione all'interno della clausola `ORDER BY` serve per ordinare i risultati:

```
1 ORDER BY titolo || città
2 --Ordina i risultati basandosi sull'unione testuale dei due campi
```

6.5.2 Clausola FROM

```
1 FROM from_item [, ...]
```

dove:

1. `table_name [[AS] alias [(column_alias [, ...])]]`
 - se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle
 - se ci sono attributi con lo stesso nome su tabelle diverse, nel `SELECT` questi attributi sono identificati antepoendo il nome della tabella e un `'.'`: `nomeTabella.nomeAttributo`
2. `(other_select) [AS] nomeRisultato [(column_alias [,...])]`
 - è una `SELECT` innestata. Il risultato della `SELECT` interna è lo schema con nome `nomeRisultato` su cui fare la `SELECT` corrente.
3. `from_item [NATURAL] join_type from_item [ON condition]`
 - è la clausola `JOIN`.
 - Metodo più sofisticato di 1) che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. `join_type` specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti

6.5.2.1 Uso di alias

```
1 SELECT P.Reddito, R.Telefono
2 FROM Reparti as R, Persona as P
3 WHERE ... ;
```

Utile quando è necessario specificare la provenienza dell'attributo, perchè nella base di dati sono presenti più attributi con lo stesso nome

6.5.2.2 Target List

```
1 SELECT P.Nome as NPersona, P.Reddito as RPersona
2 FROM Persone as P
3 WHERE P.Eta < 30
```

6.5.3 Clausola WHERE

```
1 WHERE condition
```

condition è un'espressione booleana combinando condizioni semplici C_i con gli operatori AND, OR, NOT.

1. $C_i := \text{expr} [\theta \{ \text{expr} \mid \text{const} \}]$ dove
 - $\text{expr} :=$ espressione che contiene riferimenti agli attributi delle tabelle che compaiono nella clausola FROM.
 - $\theta \in \{=, <, >, <=, >=, <>\}$
 - $\text{const} :=$ valore dei domini di base.
2. Una riga soddisfa **condition** se l'espressione è vera quando valutata con i valori della riga.
3. Una riga che non soddisfa condition è scartata dal risultato finale

```
1 SELECT Nome, Reddito
2 FROM Persone
3 WHERE Eta < 30;
```

```
SELECT Nome, Reddito FROM Persone
WHERE Eta < 30 OR Reddito > 50;
```

6.5.3.1 Operatore LIKE

```
1 WHERE attributo [NOT] LIKE 'pattern';
```

Nella clausola WHERE può apparire l'operatore LIKE per il confronto di stringhe. LIKE è un operatore di pattern matching. I pattern si costruiscono con i caratteri speciali `_` e `%`:

- `_` = 1 carattere qualsiasi.
- `%` = 0 o più caratteri qualsiasi.

```

1 SELECT Nome, Reddito
2 FROM Persona
3 WHERE Nome LIKE 'A\%a'

```

Nome	Reddito
Andrea	21
Anna	35

6.5.3.2 Operatore SIMILAR TO

L'operatore **SIMILAR TO** è un **LIKE** più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL). Esempi di componenti di espressioni regolari:

- `_` = 1 carattere qualsiasi.
- `%` = 0 o più caratteri qualsiasi.
- `*` = ripetizione del precedente match 0 o più volte.
- `+` = ripetizione del precedente match UNA o più volte.
- `{n,m}` = ripetizione del precedente match almeno n e non più di m volte.
- `[ ... ]` = ... è un elenco di caratteri ammissibili

```

1 SELECT *
2 FROM Persone
3 WHERE Nome SIMILAR TO '[AFO]'

```

Per trovare nomi che iniziano per A,F o O

6.5.3.3 Operatore BETWEEN

```

1 WHERE attributo [NOT] BETWEEN valore_iniziale AND valore_finale;

```

Nella clausola **WHERE** può apparire l'operatore **BETWEEN** per testare l'appartenenza di un valore ad un intervallo (non limitato ai valori numerici, usato anche con date e stringhe)

```

1 SELECT *
2 FROM Persone
3 WHERE Eta BETWEEN 20 AND 40

```

Nome	Età	Reddito
Andrea	27	21
Aldo	25	15
Filipo	26	30
Olga	30	41

6.5.3.4 Operatore IN

```

1 WHERE attributo [NOT] IN ( valore [, ...] );

```

Nella clausola **WHERE** può apparire l'operatore **[NOT] IN** per testare l'appartenenza di un valore ad un insieme.

```
1 SELECT *
2 FROM Persone
3 WHERE Nome NOT IN ('Andrea', 'Franco')
```

6.5.3.5 Operatore IS NULL

```
1 WHERE attributo IS [ NOT ] NULL;
```

Nella clausola WHERE può apparire l'operatore **IS [NOT] NULL** per testare se un valore è **NOT KNOWN (NULL)** o no. Il valore NULL non rappresenta lo zero o una stringa vuota, ma l'assenza di dati o un valore sconosciuto. A partire dalle versioni SQL-2 è stata adottata una logica a tre valori (true, false, unknown) e i valori nulli non sono considerati né veri, né falsi.

NON SI PUÒ usare '=' o '<>' con il valore NULL!

```
1 SELECT *
2 FROM Persone
3 WHERE Reddito IS NOT NULL
```

6.5.3.6 Operatore ORDER BY

```
1 ORDER BY attributo [{ ASC | DESC }] [, ... ];
```

La clausola ORDER BY ordina le tuple del risultato in ordine rispetto agli attributi specificati, ASCendente è lo standard.

```
1 SELECT Nome
2 FROM Persone
3 ORDER BY Nome DESC
```

6.5.3.7 Operatori di aggregazione

Sono operatori che permettono di determinare un valore considerando i valori ottenuti da una SELECT. Due tipi principali:

- COUNT.
- MAX, MIN, AVG, SUM.

```
1 SELECT MAX(Reddito)
2 FROM Persone;
```

Nota

Quando si usano gli operatori aggregati, dopo la SELECT non possono comparire espressioni che usano i valori presenti nelle singole tuple perché il risultato è sempre e solo una

tupla

1. Count

```
1 COUNT ({ * | expr | ALLexpr | DISTINCTexpr })
```

Restituisce il numero di tuple significative nel risultato dell'interrogazione. Tre casi comuni:

- **COUNT(*)** ritorna il numero di tuple nel risultato dell'interrogazione.
- **COUNT(expr)** ritorna il numero di tuple in ciascuna delle quali il valore expr è non nullo
- **COUNT(ALL expr)** include duplicati e valori nulli.
- **COUNT(DISTINCT expr)** come con **COUNT(expr)** ma con l'ulteriore condizione che i valori di expr siano distinti

2. SUM/MAX/MIN/AVG

```
1 op ({ expr | DISTINCTexpr })
```

Determinano un valore numerico (SUM/AVG) o alfanumerico (MAX/MIN) considerando le tuple significative nel risultato dell'interrogazione.

- op = SUM | AVG | MAX | MIN;
- expr è un'espressione che usa uno o più attributi e funzioni di attributi.
- DISTINCT considero solo i valori distinti.

```
1 SELECT AVG(Eta) AS Media\_Eta
2 FROM Persone;
```

6.6 Esempi di interrogazioni

6.6.1 Interrogazione con raggruppamento: Group by

Un raggruppamento è un insieme di tuple che hanno medesimi valori su uno o più attributi caratteristici del raggruppamento.

La clausola GROUP BY attr [, ...] permette di determinare tutti i raggruppamenti delle tuple della relazione risultato (tuple selezionate con la clausola WHERE) in funzione degli attributi dati.

In una interrogazione che usa GROUP BY, la clausola SELECT contiene solamente gli attributi (da 0 a tutti) utilizzati per il raggruppamento ed eventuali funzioni di aggregazione sugli altri attributi.

Visualizzare tutte le città raggruppate della tabella Studente

```
1 SELECT citta
2 FROM studente
3 GROUP BY citta
```

matricola	cognome	nome	Indirizzo	città	media
IN0001	Rossi	Marco	Via nobile	Verona	27.50
IN0002	Paoli	Paolo	Via dritta	Padova	28.6666
IN0003	Bianchi	Dante	Via storta	VERONA	18.345
IN0004	Rossi	Matteo	Via nobile	Verona	24.567
IN0005	Verdi	Luca	Via nuova	Va	28.6666
IN0006	Nocasa	Caio			

città
Padova
Verona
VERONA
Va

Visualizzare tutte le città e i cognomi raggruppati

```

1 SELECT cognome, LOWER(città)
2 FROM Studente
3 GROUP BY cognome, LOWER(città);

```

cognome	città
Verdi	Va
Rossi	verona
Paoli	padova
Nocasa	
Bianchi	verona

Nota

Nel GROUP BY non si possono usare espressioni con operatori di aggregazione.

Nota

Nella SELECT con GROUP BY non si possono specificare attributi che non sono raggruppati.

- La clausola WHERE permette di selezionare le righe che devono far parte del risultato.
- La clausola HAVING permette di selezionare i raggruppamenti che devono far parte del risultato.
- La sintassi è HAVING bool_expr, dove bool_expr è un'espressione booleana che può usare gli attributi usati nel GROUP BY e/o gli altri attributi mediante operatori di aggregazione.

Visualizzare tutte le città raggruppate che iniziano con 'V' della tabella Studente.

```

1 SELECT città
2 FROM Studente
3 GROUP BY città
4 HAVING città LIKE 'V%';

```

Città
Verona
VERONA
Va

Visualizzare tutte le città e i cognomi raggruppati con almeno due studenti con lo stesso cognome..

```

1 SELECT cognome, LOWER(città)
2 FROM Studente
3 GROUP BY cognome, LOWER(città)
4 HAVING COUNT (cognome) >1;

```

cognome	città
Rossi	verona

Si possono specificare espressioni con operatori di aggregazione su attributi non raggruppati anche nella clausola HAVING.

```
1 SELECT LOWER(città) AS "Città",
   AVG (media)::DECIMAL(5 ,2) AS "
   mediaArr."
2 FROM Studente
3 GROUP BY LOWER(città)
4 HAVING AVG(media) >23.47 ;
```

Città	mediaArr
padova	28.67
va	28.67
Verona	23.47

6.6.2 Interrogazione con JOIN

```
1 TABLE1 join_type TABLE2 ON join_condition[, ...].
```

INNER JOIN, LEFT [OUTER] JOIN, RIGHT [OUTER] JOIN e FULL [OUTER] JOIN. Dove `join_condition` è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola WHERE. Il prodotto cartesiano di due o più tabelle è un CROSS JOIN.

Interrogazioni con INNER JOIN

```
1 table1 [INNER] JOIN table2 ON join_condition[, ...]
```

Rappresenta il tradizionale θ join dell'algebra relazionale. Combina ciascuna riga r_1 di `table1` con ciascuna riga di `table2` che soddisfa la condizione della clausola ON.

6.6.3 Interrogazioni con LEFT OUTER JOIN

```
1 table1 LEFT[OUTER] JOIN table2 ON join_condition[, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga r_1 di `table1` che non soddisfa la condizione con qualsiasi riga di `table2`, si aggiunge una riga al risultato con i valori di r_1 e assegnando NULL agli altri attributi..

Il LEFT [OUTER] JOIN non è simmetrico! Con le medesime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!

6.6.4 Interrogazioni con RIGHT OUTER JOIN

```
1 table1 RIGHT[OUTER] JOIN table2 ON join_condition[, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga r_2 di `table2` che non soddisfa la condizione con qualsiasi riga di `table1`, si aggiunge una riga al risultato con i valori di r_2 e assegnando NULL agli altri attributi..

6.6.5 Interrogazioni con FULL OUTER JOIN

```
1 table1 FULL[OUTER] JOIN table2 ON join\_condition[, ...].
```

- È equivalente a: INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN.
- Non è equivalente a CROSS JOIN!

Quando usare left join rispetto a right join?

- **JOIN**: Solo i record che fanno "match" in entrambe le tabelle.
- **LEFT JOIN**: Tutto ciò che sta nella prima tabella scritta, con i NULL se manca il pezzo della seconda.
- **RIGHT JOIN**: Tutto ciò che sta nella seconda tabella scritta, con i NULL se manca il pezzo della prima.
- **FULL JOIN**: Tutti i dati di entrambe le tabelle, allineando ciò che combacia e mettendo NULL ovunque manchi una corrispondenza.

6.7 Interrogazioni nidificate e insiemistiche

Interrogazioni nidificate

SQL permette la costruzione di interrogazioni che presentano al loro interno altre interrogazioni. La nidificazione può avvenire nelle clausole **SELECT**, **FROM**, **WHERE**.

L'uso più comune è la nidificazione nella clausola WHERE

6.7.1 Clausola FROM

Con l'utilizzo di questa opzione, è possibile comporre catene di interrogazioni, in cui ciascuna interrogazione utilizza il risultato della precedente come se fosse una tabella di base

```
1 SELECT titolo, prezzoIntero
2 FROM Mostra, (
3     SELECT MAX ( prezzoIntero )
4     FROM Mostra
5 ) AS T( prezzoMax )
6 WHERE prezzoIntero = prezzoMax;
```

dove:

- **T**: alias della tabella. La query interna ha generato una tabellina "anonima", noi la chiamiamo temporaneamente con la lettera T
- **(prezzoMax)**: alias della colonna

6.7.2 Clausola WHERE

Il predicato (predicato complesso) nella clausola WHERE confronta un valore (ottenuto come espressione valutata sulla singola riga), con il risultato dell'esecuzione di un'interrogazione

SQL (in generale un insieme di valori). Occorre estendere gli operatori di confronto per poter realizzare la comparazione tra un valore e un insieme di valori

Soluzione offerta da SQL: estensione degli operatori di confronto (>, <, =, <=, >=, <>) con ANY/SOME e ALL (IN, NOT IN).

- **ANY/SOME**: specifica che la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo per la riga e **almeno uno** degli elementi restituiti dall'interrogazione.
- **ALL**: specifica che la riga soddisfa la condizione solo se **tutti gli elementi** restituiti dall'interrogazione nidificata rendono vero il confronto.
- **IN** e **NOT IN**: usati per rappresentare l'appartenenza o l'esclusione rispetto ad un insieme (identici a = ANY o <>ALL)

6.7.2.1 Operatore ANY/SOME

```
1 expression operator ANY( subquery )
2 expression operator SOME( subquery )
```

- **expression** è un'espressione che coinvolge attributi della SELECT principale.
- (**subquery**) è una **SELECT** che deve restituire UNA sola colonna;
- **operator** è un operatore di confronto, come '=', '>=', ...
- **ANY**: è vero se la condizione è vera per almeno un valore della subquery.
- **SOME** è uno sinonimo di **ANY**

Visualizzare il nome degli insegnamenti che hanno un numero di crediti inferiore alla media dell'ateneo di un qualsiasi anno accademico.

```
1 SELECT DISTINCT I.nomeins , IE.crediti
2 FROM Insegn I
3 JOIN InsErogato IE ON I.id=IE.id_insegn
4 WHERE IE.crediti < ANY (
5     SELECT AVG ( crediti )
6     FROM InsErogato
7     WHERE modulo =0
8     GROUP BY annoaccademico
9 );
```

6.7.2.2 Operatore ALL

```
1 expression operator ALL( subquery )
```

- **expression** è un'espressione che coinvolge attributi della SELECT principale.
- (**subquery**) è una **SELECT** che deve restituire UNA sola colonna;
- **operator** è un operatore di confronto, come '=', '>=', ...
- **ALL**: La condizione è vera solo se è vera per TUTTI i valori della subquery.

Trovare il nome degli insegnamenti con almeno un docente e crediti maggiori rispetto ai crediti di ciascun insegnamento del corso di

laurea con id=6. (si considerano solo occorrenze insegnamento genitore)

```

1 SELECT DISTINCT I. nomeins , IE. crediti
2 FROM Insegn I
3     JOIN InsErogato IE ON I.id = IE. id_insegn
4     JOIN Docenza D ON IE.id = D. id_inserogato
5 WHERE IE. modulo = 0 AND IE. crediti > ALL (
6     SELECT crediti
7     FROM InsErogato
8     WHERE id_corsostudi = 6 AND modulo =0
9 );

```

6.7.2.3 Operatore IN

E' un operatore utilizzabile nei predicati complessi.

```

1 ROW ( expr [ ,...]) IN ( subquery )

```

- **expr** è un'espressione costruita con un attributo della query principale. Ci possono essere una o più espressioni.
- La (**subquery**) deve restituire un numero di colonne pari al numero di espressioni in (**expr** [...]).
- I valori dell'espressioni vengono confrontati con i valori di ciascuna riga del risultato di (subquery).
- Il confronto ritorna vero se i valori sono uguali ai valori di almeno una riga della subquery.

Trovare gli impiegati che hanno stesso nome e cognome di qualcuno in un'altra azienda.

```

1 SELECT I.nome , I.cognome
2 FROM Impiegato I
3 WHERE ROW( I.nome , I.cognome ) IN (
4 SELECT I1.nome, I1.cognome FROM ImpiegatoAltraAzienda I1)

```

6.7.2.4 Operatore EXIST

E' una clausola utilizzabile nei predicati complessi

```

1 EXISTS(subquery)

```

- (subquery) è una **SELECT**.
- **EXISTS** ritorna falso se (subquery) non contiene righe; altrimenti vero.
- **EXISTS** è significativo quando nella (subquery) si selezionano righe usando i valori della riga corrente nella **SELECT** principale: **data binding**. Vale a dire se subquery è un'interrogazione dipendente dalla query esterna

Determinare i nomi degli impiegati che sono diversi tra loro ma di pari lunghezza

```

1 SELECT I. nome
2 FROM Impiegato I
3 WHERE EXISTS (
4     SELECT 1
5     FROM Impiegato I1
6     WHERE I.nome <> I1. nome AND CHAR_LENGTH( I.nome ) = CHAR_LENGTH( I1
7         . nome )
8 )

```

I.nome nella subquery è il valore di nome nella riga corrente della SELECT principale

Nota

Se si usano attributi esterni nella subquery (data binding), la subquery deve essere valutata per ogni riga della SELECT principale

Visualizzare il nome e il cognome dei docenti, escludendo i coordinatori, che hanno tenuto almeno due insegnamenti (o moduli) con più di 24 crediti nel 2010/2011

```

1 SELECT P.nome , P.cognome
2 FROM Persona P
3 WHERE EXISTS (
4     SELECT 1
5     FROM Docenza D
6     JOIN InsErogato IE ON D. id_inserogato = IE.id
7     WHERE IE. crediti > 24 AND D. coordinatore = '0'
8     AND IE. annoaccademico = '2010/2011' AND D. id_persona = P.id
9     GROUP BY D. id_persona
10    HAVING COUNT (*) >=2
11 );

```

Visualizzare il nome dei corsi di studio che nel 2006/2007 non hanno erogato insegnamenti il cui nome contiene la sottostringa 'Info'.

```

1 SELECT CS. nome FROM CorsoStudi CS
2 WHERE NOT EXISTS (
3     SELECT 1
4     FROM InsErogato IE
5     JOIN Insegn I ON IE. id_insegn = I.id
6     WHERE I.nomeins LIKE '%Info%'
7     AND IE.annoaccademico = '2006/2007'
8     AND IE.id_corsostudi = CS.id
9 )

```

Interrogazioni di tipo insiemistico

```
1 query_1 { UNION | INTERSECT | EXCEPT } [ALL] query_2
```

SQL mette a disposizione anche degli operatori insiemistici UNION, INTERSECT e EXCEPT per manipolare i risultati di più query. Gli operatori insiemistici si possono utilizzare solo al livello più esterno di una query, operando sul risultato di due o più clausole SELECT. Si possono avere sequenze di UNION/INTERSECT/EXCEPT

Gli operatori si possono applicare solo quando `query1` e `query2` producono risultati con lo stesso numero di colonne e di tipo compatibile fra loro. Tutti gli operatori eliminano i duplicati dal risultato a meno che ALL non sia stato specificato.

- **UNION** aggiunge il risultato di `query2` a quello di `query1`.
- **INTERSECT** restituisce le righe che sono presenti sia nel risultato di `query1` sia in quello di `query2`
- **EXCEPT** restituisce le righe di `query1` che non sono presenti nel risultato di `query2`. In pratica esegue la differenza insiemistica

Visualizzare i nomi degli insegnamenti e i nomi dei corsi di laurea che non iniziano per 'A' mantenendo i duplicati

```
1 SELECT nomeins
2 FROM Insegn
3 WHERE NOT nomeins LIKE 'A%'
4 UNION ALL
5 SELECT nome
6 FROM CorsoStudi
7 WHERE NOT nome LIKE 'A%'
```

Visualizzare i nomi degli insegnamenti che sono anche nomi di corsi di laurea.

```
1 SELECT nomeins
2 FROM Insegn
3 INTERSECT ALL
4 SELECT nome
5 FROM CorsoStudi;
```

Visualizzare i nomi degli insegnamenti che NON sono anche nomi di corsi di laurea.

```
1 SELECT nomeins
2 FROM Insegn
3 EXCEPT
4 SELECT nome
5 FROM CorsoStudi;
```

6.7.3 Le viste

Le viste sono tabelle virtuali il cui contenuto dipende dal contenuto delle altre tabelle della base di dati. In SQL le viste vengono definite associando un nome ed una lista di attributi al risultato dell'esecuzione di un'interrogazione. **Ogni volta che si usa una vista, si esegue la query che la definisce.** Nell'interrogazione che definisce la vista possono comparire anche altre viste. SQL **non ammette** però

- dipendenze immediate (definire una vista in termini di se stessa) o ricorsive (definire una interrogazione di base e una interrogazione ricorsiva);
- dipendenze transitive circolari (V1 definita usando V2, V2 usando V3, . . . , Vn usando V1).

```
1 CREATE [ TEMP ] VIEW nome [( col_name [, ...]) ] AS query
```

TEMP: la vista è temporanea. Quando ci si sconnette, la vista viene distrutta. È un'estensione di PostgreSQL. Nella base di dati did2014 si possono fare solo viste temporanee

- **column_name**: nomi delle colonne che compongono la vista; se non vengono specificati, sono ricavati dai nomi delle colonne restituiti dalla query, eventualmente tramite alias.
- **query** deve restituire un insieme di attributi pari e nel medesimo ordine a quello specificato con (**column_name** [, ...]) se presente.
- SQL permette che una vista sia aggiornabile solo quando ciascuna tupla della vista corrisponde a una sola tupla di ciascuna tabella di base.

Definire la vista che contiene gli insegnamenti erogati completi di nomeins e codiceins presi dalla tabella Insegn.

```
1 CREATE TEMP VIEW InsErogatiCompleti AS
2 SELECT I. nomeins , I. codiceins , IE .*
3 FROM InsErogato IE
4 JOIN Insegn I ON IE. id_insegn = I.id;
```

Si vuole determinare quali sono i corsi di studi con il massimo numero di nome di insegnamenti diversi

Prima si crea una vista:

```
1 CREATE TEMP VIEW InsCorsoStudi (Nome , NumIns) AS
2 SELECT CS.nome , COUNT ( DISTINCT I. nomeins )
3 FROM CorsoStudi CS
4 JOIN InsErogato IE ON CS.id=IE. id_corsostudi
5 JOIN Insegn I ON IE. id_insegn = I.id
6 GROUP BY CS.nome;
```

Poi la si usa:

```
1 SELECT Nome , NumIns
2 FROM InsCorsoStudi
3 WHERE NumIns = ANY (
4 SELECT MAX ( NumIns ) FROM InsCorsoStudi);
```

Laurea specialistica in Medicina e Chirurgia: 320

6.8 Indici

Gli indici sono delle **strutture dati ausiliare** che permettono di accedere ai dati di una tabella in maniera più efficiente. Essendo una struttura dati ausiliaria, deve essere sempre mantenuta aggiornata in base al contenuto della tabella in cui è definita. Il costo di aggiornamento può essere significativo quando ci sono molti indici definiti sulla medesima tabella. Gli indici quindi devono essere definiti considerando la loro utilità.

Si consideri il seguente schema fisico

```

1  InsErogato(id, id_insegn, id_corsostudi, id_discriminante,
    annoaccademico, id_facolta, modulo, nomemodulo, discriminantemodulo
    , crediti, programma)
2  Insegn(id, nomeins, codiceins)
3  Discriminante(id, nome, descrizione)
4  CorsoStudi(id, nomecorsostudi, sede, durataAnni)
5  CorsoInFacolta(id, id_corsostudi, id_facolta)
6  Facolta(id, nome)

```

Query SENZA indici

Si consideri la query:

```

1  SELECT id, nomeins FROM Insegn WHERE nomeins='Algoritmi';

```

Il DBMS deve fare una scansione sequenziale della tabella Insegn rispetto nomeins per estrarre le righe con valore uguale a 'Algoritmi';

Il comando \timing attiva/disattiva la visualizzazione del tempo di pianificazione + calcolo di una query. Il tempo è visualizzato subito dopo la visualizzazione del risultato della query. \timing dà un'idea del tempo necessario per eseguire una query.

```

1  =>\timing
2  Timing is ON.
3  => SELECT id , nomeins
4  FROM Insegn
5  WHERE nomeins='Algoritmi';
6
7
8  id | nomeins
9  ----+-----
10  5441 | Algoritmi
11
12  (1 row)
13  TIME: 2.437 ms

```

Il tempo è di 2 millisecondi circa

6.8.1 Creazione indice

```

1 CREATE INDEX [ nome] ON nomeTabella [ USING method ]
2 ({ nomeAttr | ( expression ) } [ ASC | DESC ] [, ...])
3 CREATE INDEX Nome Indice ON NomeTabella (ListaAttributi)

```

- **method** è il tipo di indice;
- **nomeAttr** o **expression** indicano su quali espressione di attributi si deve creare indice.
- **ASC/DESC** specifica se l'indice è ordinato in modo ascendente o discendente.
- **ALTER INDEX** e **DROP INDEX** permettono di modificare/rimuovere indici creati precedentemente.

Un DBMS crea gli indici in modo automatico solo per attributi dichiarati PRIMARY KEY (**indice primario**).

Per tutti gli altri attributi si deve fare una dichiarazione esplicita di **CREATE INDEX**.

```

1 ANALYZE nomeTabella;

```

Il comando ANALYZE[nomeTabella] aggiorna le statistiche circa il contenuto delle tabelle (e indici). È eseguito in modo automatico dal sistema a intervalli regolari. L'ottimizzatore di query usa queste statistiche per decidere quando usare gli indici.

Query CON indici

Per rendere più veloce l'esecuzione della query:

```

1 SELECT id, nomeins FROM Insegn WHERE nomeins='Algoritmi';

```

un indice sull'attributo nomeins potrebbe essere utile perché nomeins è l'attributo usato per selezionare le righe. Query CON

```

1 => CREATE INDEX nomeins_index ON Insegn( nomeins);
2 => ANALYZE Insegn;
3 => SELECT id, nomeins FROM Insegn WHERE nomeins='Algoritmi';
4
5 id | nomeins
6 ----+-----
7 5441 | Algoritmi
8 TIME: 0.587 ms

```

Il tempo di esecuzione è passato da 2.4337 ms a 0.587 ms.

Nota

Nota: Non è garantito che il tempo di esecuzione sia sempre uguale perché il server è multitask. I tempi di queste slide sono stati determinati con il server completamente dedicato

6.8.2 Caso pratico

Si consideri la seguente query senza Indici

```

1 SELECT DISTINCT I.nomeins
2 FROM CorsoStudi CS
3 JOIN InsErogato IE ON CS.id = IE.id_corsostudi
4 JOIN Insegn I ON I.id = IE.id_insegn
5 WHERE IE.annoaccademico = '2009/2010'
6 AND CS.nome = 'Laurea in Informatica';
7 ...
8 (26 ROWS )
9 TIME : 40.975 ms

```

Quanto è possibile rendere più veloce la query definendo degli indici?

Una possibilità è definire gli indici su tutti gli attributi usati per fare i JOIN o le selezioni

```

1 CREATE INDEX cs_id ON CorsoStudi(id);
2 CREATE INDEX i_id ON Insegn(id);

```

Nota

Gli indici sugli attributi PRIMARY KEY (id nel nostro caso) solitamente sono già presenti! In did2014small sono stati tolti per fare gli esperimenti

Si consideri la creazione degli indici:

```

1 CREATE INDEX ie_id_corsostudi ON Inserogato( id_corsostudi);
2 CREATE INDEX ie_id_insegn ON Inserogato( id_insegn);
3 CREATE INDEX ie_aa ON Inserogato( annoaccademico);
4 CREATE INDEX cs_nome ON Corsostudi( nome );
5 ANALYZE Corsostudi;
6 ANALYZE Inserogato;
7 ANALYZE Insegn;

```

Non tutti gli indici sono necessari, ma per ora accettiamo il costo di crearli tutti. Importante! Forziamo il DBMS ad aggiornare le statistiche con il comando ANALYZE dopo la creazione degli indici.

Con la creazione degli indici, la query richiede un tempo inferiore:

```

1 SELECT DISTINCT I.nomeins
2 FROM Corso StudiCS
3 JOIN InsErogato IE ON CS.id= IE.id_corsostudi
4 JOIN InsegnI ON I.id= IE.id_insegn
5 WHERE IE.annoaccademico= '2009/2010'
6 AND CS.nome= 'Laurea in Informatica';
7 ...
8 (26 ROWS )
9 TIME : 5.201 ms

```

Il tempo di esecuzione è passato da 40 ms a 5 ms.

6.8.3 Tipi di indice

PostgreSQL ammette diversi tipi di indici: B-tree, hash, GiST, SP-GiST, GIN e BRIN. Sintassi per specificare anche il tipo:

```
1 CREATE INDEX nome ON nomeTab USING type(COLUMN);
```

6.8.3.1 B-tree

Se nel CREATE INDEX non si specifica il tipo di indice, il sistema crea un indice di tipo B-tree. L'ottimizzatore di query considera un indice B-tree ogni volta che l'attributo indicizzato è coinvolto in un confronto usando uno degli operatori: <, <=, =, >=, >. Un indice B-tree può essere considerato anche quando ci sono **BETWEEN**, **IN**, **IS NULL**, **IS NOT NULL** e **LIKE** 'abc%'.

Con la localizzazione (it_IT.UTF-8), la comparazione di stringhe con operatori diversi da '=' e con **LIKE** ha regole diverse (minuscole/maiuscole) rispetto a UTF8. Un indice su attributo VARCHAR in una base di dati con LC_LOCALE = it_IT.UTF-8 (come le basi di dati in dbserver) dovrebbe essere dichiarato come:

```
1 CREATE INDEX nome ON nomeTab(nomeAttrvarchar_pattern_ops);
```

se si vuole che l'indice sia usato anche quando il confronto usa <,>,<>, LIKE. La parola chiave **varchar_pattern_ops** abilita l'uso degli operatori rispettando le regole imposte dal locale (it_IT.UTF-8).

6.8.3.2 Hash

Il tipo **hash** ha un uso limitato perché può essere usato solo quando i confronti sonodi eguaglianza. La sua gestione poi è più complicata in basi di dati replicate o in caso di crash: il gruppo di PostgreSQL ne sconsiglia l'uso. Gli altri tipi di indici sono indicati per particolari strutture dati. Per esempio, GiST è indicato quando un attributo è di tipo geometrico bi-dimensionale(tipo non standard).

6.8.3.3 Multi-attributo

Se si hanno query che hanno condizioni su coppie (terne, ecc) di attributi di una tabella, un indice multi-attributo definito usando la coppia (terna, ecc) di attributi potrebbe essere più utile rispetto gli indici sui singoli attributi.

Creazione indice multi-attributo

```
1 CREATE INDEX ie_aa_idcs ON
2
3 Inserogato( annoaccademico, id_corsostudi);
4 ANALYZE Inserogato;
5
6 SELECT I.nomeins, I.codiceins
7 FROM InsegnI JOINInserogatoIE ONI.id= IE. id_insegn
8 WHERE IE. annoaccademico= '2006/2007 ';
```



```

9 ANDIE. id_corsostudi= 4;
10
11 ... (46 ROWS )
12 TIME : 1.910 ms

```

NON sempre gli indici multi-attributo possono essere usati: per esempio in espressioni con OR non è possibile.

```

1 SELECT I. nomeins, I. codiceins
2 FROM InsegnI JOIN InsErogatoIE ON I.id= IE. id_insegn
3 WHERE IE.annoaccademico= '2006/2007' OR IE.id_corsostudi= 4;
4 ... (5334 ROWS )

```

- Non c'è il tempo di esecuzione perché il risultato è diverso e quindi non comparabile.
- In questa query l'indice `ie_aa_idcs` NON è usato perché la disgiunzione **OR** rende impossibile di fatto il suo uso.
- Nemmeno gli indici singoli `ie_id_corsostudi` e `ie_aa` sono usati.
- L'ottimizzatore esegue una scansione di tutta la tabella `Insegn`.

6.8.3.4 Indici di espressioni

È possibile definire indici anche usando espressioni di attributi.

```

1 CREATE INDEX nome ON nomeTabella(expression);

```

dove `expression` è una espressione su uno o più attributi.

Query con indici definiti usando espressioni:

```

1 CREATE INDEX ins_lower_nonins ON Insegn
2 ( LOWER( nomeins) varchar_pattern_ops);
3 ANALYZE Insegn;
4 SELECT nomeins FROM Insegn
5 WHERE LOWER(nomeins) LIKE 'algoritmi %';
6 ... (4 ROWS )
7 TIME : 1.796 ms

```

6.8.4 Costo e Pratiche d'uso

Gli indici costano tempo e memoria. Se si considera solo il tempo, il costo maggiore è mantenere gli indici aggiornati. Per ogni operazione di INSERT, UPDATE, DELETE su una tabella con indici, il DBMS deve aggiornare anche gli indici della tabella.

Regola pratica: definire gli indici in base alle query più frequenti.

Il comando PostgreSQL `\di` visualizza l'elenco degli indici definiti in una base di dati.

Il comando PostgreSQL `\d nomeTabella` visualizza la definizione di tabella e degli indici associati.

Perchè questo?

Un DBMS ha un ottimizzatore di query che determina un piano per eseguire una data query nel minor tempo possibile. Il comando EXPLAIN permette di vedere il piano di esecuzione di una query che l'ottimizzatore determina senza eseguire la query. Un piano di esecuzione di una query è un albero di nodi di esecuzione.

Le foglie sono nodi di scansione: l'esecuzione di questi nodi restituiscono indirizzi di righe della tabella.

Struttura

Esistono differenti tipi di scansioni: **sequenziali**, **indicizzate** e **mappate su bit**.

- Se una query contiene **JOIN**, **GROUP BY** o **ORDER BY** o altre operazioni sulle righe, **allora ci saranno altri nodi di esecuzione sopra le foglie nell'albero**.
- L'output di EXPLAIN ha una riga per ciascun nodo nell'albero di esecuzione dove viene indicato il tipo di operazione e una stima del costo di esecuzione.
- Ulteriori proprietà del nodo possono essere mostrate con una riga indentata subito sotto.
- La prima riga dell'output ha la stima del costo totale di esecuzione della query. Questo è il costo che l'ottimizzatore cerca di minimizzare.

```

1 EXPLAIN SELECT* FROM Insegn;
2 QUERY PLAN
3 -----
4 SeqScan ON insegn(cost=0.0..185.69 ROWS=8169 width=63)

```

- 0.0 è il costo iniziale, per produrre la prima riga(in termini di numero di accessi alla memoria secondaria).
- 185.69 è il costo totale, per produrre tutte le righe.
- 8169 è il numero totale di righe del risultato(Il numero totale di righe non è sempre il numero totale di righe valutate dall'esecutore).
- 63 è la dimensione, in byte, di ciascuna riga.

6.8.4.1 Explain con due nodi

```

1 EXPLAIN SELECT* FROM Insegn WHERE id < 1000;
2 QUERY PLAN
3 -----
4 Bitmap Heap Scan ON insegn(cost=18.60..132.79 ROWS=815...)
5 Recheck Cond: (id < 1000)
6 -> Bitmap INDEX Scan ON i1 (cost=0..18.39 ROWS=815 w =0)
7 INDEX Cond: (id < 1000)

```

Prima viene eseguito il nodo foglia (Bitmap INDEX Scan): grazie all'indice B-tree, l'ottimizzatore determina un vettore di indirizzi di righe da considerare (in un B-tree la chiave cercata k non è pari a nessuna chiave K_i ma è compresa tra K_i e K_{i+1} , può essere presente nel sottoalbero P_i). Tale vettore viene poi passato al nodo padre (Bitmap Heap Scan) che carica le righe ed esegue la selezione finale ricontrollando che $id < 1000$.

6.8.4.2 Explain con due nodi – AND con un solo indice

```

1 DROP INDEX ins_nomeins;
2 EXPLAIN SELECT* FROM Insegn WHERE id <1000 AND nomeins LIKE 'A1%';
3 QUERY PLAN
4 -----
5 Bitmap Heap Scan ON Insegn(cost=18.40..134.62 ROWS=1...)
6 Recheck Cond: (id < 1000)
7 Filter: (( nomeins)::TEXT~~ 'A1% '::TEXT)
8 -> Bitmap INDEX Scan ON i1 (cost=0.00..18.39 ROWS=815 width=0)
9 INDEX Cond: (id < 1000)

```

Il nodo Bitmap Heap Scan carica le righe indicate dall'indice e, per ciascuna, ricontrrolla che $id < 1000$, e la seleziona se nomeins inizia con 'A1'.

6.8.4.3 Esempi d'uso

Esempio di EXPLAIN con uso di due indici sulla stessa tabella

```

1 CREATE INDEX nomeIdx ON Insegn(nomeins varchar_pattern_ops);
2 ANALYZE Insegn;
3 EXPLAIN SELECT* FROM Insegn WHERE id<1000 AND nomeins LIKE 'A1%';
4 QUERY PLAN
5 -----
6 Bitmap Heap Scan ON insegn(cost=15.30..22.58 ROWS=4 width=63)
7 Recheck Cond: (id < 1000)
8 Filter: (( nomeins)::TEXT~~ 'A1% '::TEXT)
9 -> BitmapAnd(cost=15.30..15.30 ROWS=4 width=0)
10 -> Bitmap INDEX Scan ON nomeidx(cost=0..2.65 ROWS=37 width=0)
11 INDEX Cond: (((nomeins)::TEXT ~>=~ 'A1'::TEXT) AND
12 ((nomeins)::TEXT~<~ 'Am'::TEXT))
13 -> Bitmap INDEX Scan ON i1 (cost=0.00..12.40 ROWS=815 width=0)
14 INDEX Cond: (id < 1000)

```

- Ciascuna foglia scansiona un indice. Il padre delle foglie (BitmapAnd) fa l'intersezione degli indirizzi delle righe trovate.
- La root (Bitmap Heap Scan) carica le righe ed ricontrrolla che le condizioni siano soddisfatte.

Esempio di EXPLAIN con uso di due indici sulla stessa tabella

Esempio di EXPLAIN con uso di due indici sulla stessa tabella

```

1 EXPLAIN SELECT* FROM Insegn WHERE id <1000 OR nomeins LIKE 'A1%';
2 QUERY PLAN
3 -----
4 Bitmap Heap Scan ON insegn(cost=15.47..132.25 ROWS=850 width=63)
5 Recheck Cond: ((id < 1000) OR((nomeins)::TEXT~~ 'A1% '::TEXT))
6 Filter : ((id < 1000) OR((nomeins)::TEXT~~ 'A1% '::TEXT))
7 -> BitmapOr(cost=15.47..15.47 ROWS=852 w=0)
8 -> Bitmap INDEX Scan ON i1 (cost=0.00..12.40 ROWS=815 width=0)
9 INDEX Cond: (id < 1000)
10 -> Bitmap INDEX Scan ON nomeidx(cost=0.00..2.65 ROWS=37 width=0)
11 INDEX Cond: (((nomeins)::TEXT~>=~ 'A1'::TEXT) AND
12 ((nomeins)::TEXT~<~ 'Am'::TEXT))

```

Il padre delle foglie (BitmapOr) esegue l'unione degli indirizzi delle righe trovate.

Esempio di EXPLAIN con planner che decide diversamente cambiando condizione

```

1 EXPLAIN SELECT* FROM Insegn WHERE id <1000 AND nomeins LIKE '%A1';
2 QUERY PLAN
3 -----
4 Bitmap Heap Scan ON insegn(cost=12.40..128.62 ROWS=1 width=63)
5 Recheck Cond: (id < 1000)
6 Filter: ((nomeins)::TEXT~~ '%A1'::TEXT)
7 -> Bitmap INDEX Scan ON i1 (cost=0.00..12.40 ROWS=815 width=0)
8 INDEX Cond: (id < 1000)

```

La condizione nomeins LIKE '%A1' rende impossibile l'uso dell'indice.

Esempio di EXPLAIN di un JOIN: join mediante hash table

```

1 EXPLAIN SELECT* FROM InsegnI JOIN InserogatoIE ON ie.id_insegn= i.id
   WHERE ie.annoaccademico= '2013/2014';
2 QUERY PLAN
3 -----
4 Hash JOIN(cost=287.80..6328.90 ROWS=5155 width=641)
5 Hash Cond: (ie. id_insegn= i.id)
6 -> Seq Scan ON inserogatoie(cost =0.00..5970.21 ROWS =5155 w =578)
7 Filter : ((annoaccademico)::TEXT= '2013/2014'::TEXT)
8 -> Hash (cost =185.69..185.69 ROWS =8169 width=63)
9 -> Seq Scan ON insegni (cost =0.00..185.69 ROWS =8169 w =63)

```

Esempio di EXPLAIN di un JOIN: join mediante nestedloop

```

1 EXPLAIN SELECT* FROM InsegnI JOIN InserogatoIE ON ie.id_insegn> i.id
   WHERE ie. annoaccademico= '2013/2014';
2 QUERY PLAN
3 -----
4 Nested Loop (cost=0.28..393703.79ROWS=14037065 width=641)
5 -> SeqScanONinserogatoie(cost=0.00..5970.21 ROWS=5155 w=578)
6 Filter: (( annoaccademico)::TEXT= '2013/2014':: TEXT)
7 -> INDEX Scan USING i1 ON insegni (cost =0.28..47.99 ROWS=2723 w=63)
8 INDEX Cond: (ie. id_insegn> id)

```

Ottimizzazione della query precedente:

```

1 CREATE INDEX ie_aa ON Inserogato(annoaccademico varchar_pattern_ops);
2 ANALYZE Inserogato;
3 EXPLAIN SELECT* FROM InsegnI JOIN InserogatoIE ON ie. id_insegn> i.id
   WHERE ie.annoaccademico= '2013/2014';
4 QUERY PLAN
5 -----
6 Nested Loop ( cost =82.65.. 391932.20ROWS =14037065 width =641)
7 -> Bitmap Heap Scan ON inserogatoie(cost=82..4198 rows=5155 w=578)
8 Recheck Cond: ((annoaccademico)::TEXT= '2013/2014'::TEXT)
9 -> Bitmap INDEX Scan ON ie_aa(cost=0.00..81.08 ROWS=5155 w=0)
10 INDEX Cond: ((annoaccademico)::TEXT= '2013/2014':: TEXT)
11 -> INDEX Scan USING i1 ON insegni(cost =0.28..48 ROWS =2723 w =63)

```

```
12 INDEX Cond : (ie. id_insegn > id)
```

Altro tipo di JOIN: join mediante merge join

```
1 EXPLAIN SELECT *FROM t1, t2
2 WHERE t1.unique1 < 100 AND t1.unique2 = t2.unique2 ;
3 QUERY PLAN
4 -----
5 Merge JOIN (cost=198.11..268.19 ROWS=10 width=488)
6 Merge Cond: (t1.unique2 = t2.unique2 )
7 -> INDEX Scan USING t1_unique2 ON t1 (cost =0..656 ROWS =101..)
8 Filter : ( unique1 < 100)
9 -> Sort(cost =197.83..200.33 ROWS =1000..)
10 Sort KEY: t2.unique2
11 -> SeqScanON t2 (cost =0.00..148.00 ROWS =1000..)
```

6.8.5 Explain versione avanzata

Il comando **EXPLAIN ANALYZE** mostra il piano di esecuzione, esegue la query senza registrare eventuali modifiche e mostra, infine, una stima verosimile dei tempi di esecuzione

Explain con esecuzione

```
EXPLAIN ANALYZE SELECT *FROM Insegn WHERE id <1000 AND nomeins LIKE 'Algo %';
QUERY PLAN
-----
Bitmap Heap Scan ON insegn (cost=18.40..134.62 ROWS=1..)
  (actual TIME=1.001..1.605 ROWS=1 loops=1)
  Recheck Cond: (id < 1000)
  Filter: (nomeins ~~ 'Algo %')
  ROWS Removed BY Filter: 814
  Heap Blocks: exact=90
  -> Bitmap INDEX Scan ON insegn_pkey (cost=0..18 ROWS=815)
    (actual TIME=0.928..0.928 ROWS=815 loops=1)
    INDEX Cond: (id < 1000)
Planning TIME: 1.243 ms
Execution TIME: 2.234 ms
```

- Per ogni nodo del piano: dettaglio righe rimosse, memoria usata e tempi di esecuzione in ms.
- Alla fine: stima tempo di pianificazione ed di esecuzione.

Explain con esecuzione di una query pesante

```
EXPLAIN ANALYZE SELECT * FROM insegn I, interrogato IE
WHERE IE.annoaccademico = '2013/2014' AND IE.id_insegn > I.id;
```

QUERY PLAN

```
-----
Nested Loop (cost=0..397132 ROWS=14148708 width=638)
  (actual TIME=0.111..18713.811 ROWS=24457113 loops=1)
  -> Seq Scan ON interrogato (cost=0..6059 ROWS=5196)
    (actual TIME=0.070..41.651 ROWS=5155 loops=1)
    Filter: (annoaccademico = '2013/2014')
    ROWS Removed BY Filter: 62862
  -> INDEX Scan USING insegn_pkey ON insegn (cost=0..48.03 ROWS=2723)
    (actual TIME=0.010..1.790 ROWS=4744 loops=5155)
    INDEX Cond: (IE. id_insegn > id)
Planning TIME: 0.612 ms
Execution TIME: 21471.997 ms
```

- Il valore di loops indica per quante volte viene eseguito il nodo.
- Spesso il numero di righe dopo cost è diverso dal numero di righe dopo actualTIME. Quest'ultimo è più corretto.

6.9 Introduzione al controllo di concorrenza in SQL

Una transazione SQL è una sequenza di istruzioni SQL che può essere eseguita in concorrenza con altre transazioni in modo isolato. Se si implementasse un isolamento completo, il grado di parallelismo sarebbe limitato. Si accettano quindi diversi livelli di isolamento per favorire un maggior grado di parallelismo. I diversi livelli di isolamento possono determinare delle anomalie di esecuzioni:

- **perdita di aggiornamento** (lost update)
- **lettura sporca** (dirty read)
- **letture inconsistenti** (non-repeatable read)
- **aggiornamento fantasma** (ghost update)
- **inserimento fantasma** (phantom update)

In PostgreSQL è prevista un'altra anomalia che nel testo di teoria non è considerata: **mancata serializzazione** (serialization anomaly). Il risultato di un gruppo di transazioni (nessun abort) è inconsistente con tutti gli ordini di esecuzioni seriali delle stesse. Di seguito viene mostrato come si introducono i livelli di isolamento implementati in PostgreSQL e come si possono attivare

6.9.1 Transazioni

In PostgreSQL una transazione ben formata è così composta:

- inizia con il comando **BEGIN TRANSACTION** o **START TRANSACTION**;
 - termina con un **END TRANSACTION**;
 - nel mezzo vengono eseguiti con il comando **COMMIT** (per confermare tutte le istruzioni della transazione) o con il comando **ROLLBACK** (per annullare tutte le operazioni).
- Transazione passaggio importo da

Transazione: passaggio importo da un conto ad un altro

```

1 BEGIN;
2 UPDATE accounts SET balance = balance -100.0 WHERE name = 'Alice';
3 UPDATE accounts SET balance = balance +100.0 WHERE name = 'Bob';
4 COMMIT;

```

Transazione: passaggio importo da un conto ad un altro

```

1 BEGIN;
2 UPDATE accounts SET balance = balance -100.0 WHERE name = 'Alice';
3 UPDATE accounts SET balance = balance +100.0 WHERE name = 'Bob';
4 ABORT

```

Alice ha ancora l'importo originale nel suo conto!

6.9.1.1 Controllo concorrenza funzionamento

PostgreSQL mantiene la consistenza dei dati usando un modello multi-versione (Multiversion Concurrency Control (MVCC)):

Le transizioni in PostgreSQL sono gestite come:

- Ciascuna transazione vede un'istantanea della base di dati.
- Le letture su questa istantanea sono sempre possibili e non sono mai bloccate anche se ci sono altre transazioni che stanno modificando la base di dati.
- Le scritture possono essere sospese. Una scrittura è sospesa quando, in parallelo, un'altra transazione (non ancora chiusa) ha modificato la sorgente dei dati che si vuole aggiornare.
- Al COMMIT, il sistema registra sempre l'istantanea aggiornata come nuova base di dati.

In questo modo ci sono meno conflitti e una performance più ragionevole in ambito multiutente

Esempio di sessioni concorrenti

Si consideri la tabella *Studente* con il seguente contenuto:

```
SELECT matricola, cognome, città FROM Studente;
```

Matricola	Cognome	Città
IN0001	Rossi	Verona
IN0002	Paoli	Padova
IN0003	Bianchi	VERONA
IN0004	Rossi	Verona
IN0005	Verdi	Va
IN0006	Nocasa	

Esempio di sessioni concorrenti			
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN;	1
2	SELECT * FROM STUDENTE; ... -- <i>Tabella iniziale</i>		
3	UPDATE Studente SET città = 'Vicenza' WHERE matricola = 'IN0006'; UPDATE 1	SELECT città FROM STUDENTE 3 WHERE matricola = 'IN0006'; città -----	3
4	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa Vicenza	UPDATE Studente SET città = 'VI' WHERE matricola = 'IN0006'; -- <i>esecuzione sospesa. La tabella è bloccata dall'istante 3 in T1</i>	4
5	COMMIT;		5
6	continua...	UPDATE 1 -- <i>il COMMIT di T1 ha sbloccato!</i>	6

Esempio di sessioni concorrenti			
Transazione 1		Transazione 2	
7	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa Vicenza		7
8		COMMIT;	8
9	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa VI	SELECT matricola, cognome, città FROM STUDENTE WHERE matricola = 'IN0006'; matricola cognome città -----+-----+----- IN0006 Nocasa VI	9

6.9.2 Livelli di isolamento

I 4 livelli di isolamento offerti da PostgreSQL 11.3 sono, in ordine decrescente di isolamento:

1. **Serializable**: Garantisce che un'intera transazione è eseguita in un qualche ordine sequenziale rispetto ad altre transazioni: completo isolamento da transazioni concorrenti.
 - Anomalie possibili: nessuna!
2. **Repeatable Read**: Garantisce che i dati letti durante la transazione non cambieranno a causa di altre transazioni: rifacendo la lettura dei medesimi dati, si ottengono sempre gli stessi.
 - Questo livello è più restrittivo del livello standard in SQL 'Repeatable Read'.

- Anomalie possibili: mancata serializzazione (rispetto a quanto visto a teoria questo livello di isolamento risolve anche l'anomalia di inserimento fantasma).
3. **Read Committed**: Garantisce che qualsiasi SELECT di una transazione veda solo i dati confermati (COMMITTED) prima che la SELECT inizi.
 - È il livello di isolamento di default usato da PostgreSQL.
 - Anomalie possibili: lettura inconsistente, aggiornamento fantasma, inserimento fantasma e mancata serializzazione.
 4. **Read Uncommitted**: In PostgreSQL (già da 11.3) è implementato come Read Committed. Quindi NON esiste questo livello di isolamento.

Il cambio di livello di isolamento si effettua con SET TRANSACTION appena dopo il BEGIN della transazione

6.9.2.1 Sintassi

```
1 SET TRANSACTION transaction_mode;
2 // dove transaction_mode e' uno tra
3 ISOLATION LEVEL { SERIALIZABLE | REPEATABLE READ | READ COMMITTED |
4 READ UNCOMMITTED }
5 | READ WRITE | READ
```

modo standard

```
1 BEGIN;
2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
3 COMMIT;
```

modo compatto

```
1 BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
2
3 COMMIT;
```

6.9.2.2 Livello di isolamento: Read Committed

- È il livello di default, quindi è sufficiente BEGIN;
 - SELECT vede solo i dati registrati (COMMITTED) in altre transazioni e quelli modificati da eventuali comandi precedenti nella medesima transazione.
 - UPDATE, DELETE vedono i dati come SELECT. Inoltre, se i dati che devono essere aggiornati sono stati modificati ma non registrati in transazioni concorrenti, il comando deve:
1. attendere il COMMIT o ROLLBACK della transazione dove è stato fatto il cambio
 2. riesaminare le righe selezionate per verificare che soddisfano ancora i criteri del comando.

Caso Lettura inconsistente		id (serial)	hit (integer)
Si assuma che esista la tabella W:		1	9
Si considerino le seguenti transazioni concorrenti		2	10
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN;	1
2	SELECT hit FROM W WHERE hit <=9; 9		2
3		UPDATE W SET hit =10 WHERE hit = 9;	3
4		COMMIT ;	
5	SELECT hit FROM W WHERE hit <=9; 9		
6	COMMIT;		

T1 ha due letture diverse della stessa tabella senza averla modificata.

Caso Aggiornamento Fantasma		id (serial)	hit (integer)
Si assuma la tabella W e che esista un vincolo di programma $SUM(hit) \leq 20$.		1	9
		2	10
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN;	1
2	DO \$\$ DECLARE n INTEGER;		2
3	BEGIN		3
4	n := (SELECT hit FROM W WHERE id =1); -- n=9		4
5		UPDATE W SET hit = 8 WHERE id =1;	5
6		UPDATE W SET hit = 12 WHERE id =2;	6
7		COMMIT;	7
8	n := n + (SELECT hit FROM W WHERE id =2); -- n=12		8
9	IF n > 20 THEN RAISE 'La somma è %', n; END IF;		9

La base di dati soddisfa sempre il vincolo, ma per T1 no. T1 ha perso un aggiornamento!

Livello di isolamento Read Committed

Read Committed: caso più articolato		id (serial)	hit (integer)
Transazione 1		1	9
Transazione 2		2	10
1	BEGIN;		1
2	...		2
3	UPDATE W SET hit=hit +1; UPDATE 2		3
4	...		4
5	...		5
6	COMMIT ;		6
7			7

DELETE non cancella la tupla id=2 selezionata al passo 4!
 La tupla, infatti, viene aggiornata da UPDATE; UPDATE blocca DELETE fino al COMMIT.
 DELETE, quindi, deve riesaminare (solo) la tupla id=2.
 Al riesame, la clausola WHERE hit=10 non vale più. Tupla non cancellata.

6.9.2.3 Livello di isolamento Repeatable Read

Differisce da 'Read Committed' per il fatto che i comandi di una transazione vedono sempre gli stessi dati. PostgreSQL associa alla transazione una istantanea della base di dati all'esecuzione del suo primo comando.

- 'Repeatable Read' in PostgreSQL è più stringente di quanto richiesto dallo standard SQL.
- Due **SELECT** identiche successive vedono sempre gli stessi dati.
- **UPDATE e DELETE** vedono i dati come SELECT. Se i dati che devono essere aggiornati sono stati modificati ma non registrati in transazioni concorrenti, il comando deve attendere il **COMMIT/ROLLBACK** della transazione dove è stato fatto il cambio e riesaminare le righe selezionate.
- In caso di **ROLLBACK, UPDATE e DELETE** possono procedere. In caso di **COMMIT**, i dati sono cambiati, quindi UPDATE e DELETE vengono bloccati con l'errore **ERROR**: «could NOT serialize access due to concurrent UPDATE».

6.9.2.4 Livello di isolamento: Repeatable Read

'Repeatable Read': più restrittivo di 'Committed Read'		id (serial)	hit (integer)
Si assuma che esista la tabella W:		1	9
Si considerino le seguenti transazioni concorrenti		2	10
Transazione 1		Transazione 2	
1	BEGIN;	BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;	1
2	UPDATE W SET hit=hit +1; UPDATE 2		2
3		DELETE FROM W WHERE hit =10;	3
4		COMMIT ;	4
5	COMMIT;	ERROR : could NOT serialize access due to concurrent UPDATE	5

Cattura tutte anomalie tranne mancata serializzazione. Se si usa questo livello, si deve prevedere la possibilità di transazioni abortite per aggiornamenti concorrenti.

Repeatable Read: anomalia 'mancata serializzazione'		id (serial)	hit (integer)
Repeatable Read ammette l'anomalia mancata serializzazione.		1	9
		2	10
Transazione 1		Transazione 2	
1	BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;	BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;	1
2	INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1		2
3		INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1	3
4	COMMIT;	COMMIT;	4

La tabella contiene due valori 11 e non 11 e 12. Il risultato ottenuto non è consistente con nessun ordine di esecuzione delle due transizioni.

Figura 6.2: anomalia: mancata serializzazione

È il più **restrittivo**: nessuna anomalia è permessa. Se si usa questo livello, si deve prevedere la possibilità di transazioni abortite per aggiornamenti concorrenti (come nel caso di 'Repeatable Read').

		id (serial)	hit (integer)
Serializable: anomalie di serializzazione non sono possibili!		1	9
		2	10
Transazione 1		Transazione 2	
1	BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;	BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;	1
2	INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1		2
3	COMMIT;	INSERT INTO W(hit) SELECT MAX (hit)+1 FROM W; INSERT 0 1	3
4		COMMIT;	4
		ERROR : could NOT serialize access due to READ/WRITE dependencies among Transactions DETAIL : Reason code : Canceled ON identification AS a pivot, during COMMIT attempt. HINT: The TRANSACTION might succeed if retried.	

Un metodo pratico per decidere se è necessario il livello serializable è: verificare se la selezione delle righe di un UPDATE/INSERT/DELETE in una transizione può essere modificata in una transizione concorrente (che può aggiungere/togliere o modificare le righe di interesse).

6.9.2.5 Serializable: avvertenze

- L'uso di transazioni 'Serializable' rende più semplice lo sviluppo di programmi SQL: è sufficiente dimostrare che una transazione, da sola, determina sempre uno stato corretto per avere la garanzia che essa continuerà ad essere corretta anche in ambiente concorrente dichiarandola 'Serializable'.
- Dall'altra parte però dichiarare tutte le transazioni 'Serializable' determina, in generale, una limitazione alle prestazioni (throughput) di un DMBS.

6.9.2.6 Cosa fare in caso di failure:

- È fondamentale prevedere e gestire gli errori di concorrenza che possono occorrere con livello di isolamento 'Repeatable Read' o 'Serializable'.
- Se una transizione fallisce, il codice di fallimento è **SQLSTATE = '40001'**.
- SQLSTATE è una variabile di ambiente che si può leggere sia da programmi interni sia da programmi esterni.
- In caso di errore, si deve semplicemente ritentare l'esecuzione della transazione.
- Il costo computazionale di rieseguire una transazione è solitamente meno oneroso di quello che si avrebbe se si gestissero le transazioni usando i lock espliciti come, ad esempio, SELECT FOR UPDATE e SELECT FOR SHARE

6.9.3 Lock Espliciti

PostgreSQL permette di attivare lock espliciti di tabelle e anche di righe di tabelle. Lock espliciti dovrebbero essere gestiti a livello di applicazione quando il modello MVCC non garantisce il comportamento richiesto (soprattutto a livello di prestazioni). In questo corso i lock espliciti non sono considerati. Si rimanda al capitolo 13.3 del manuale di PostgreSQL per chi volesse approfondire l'argomento